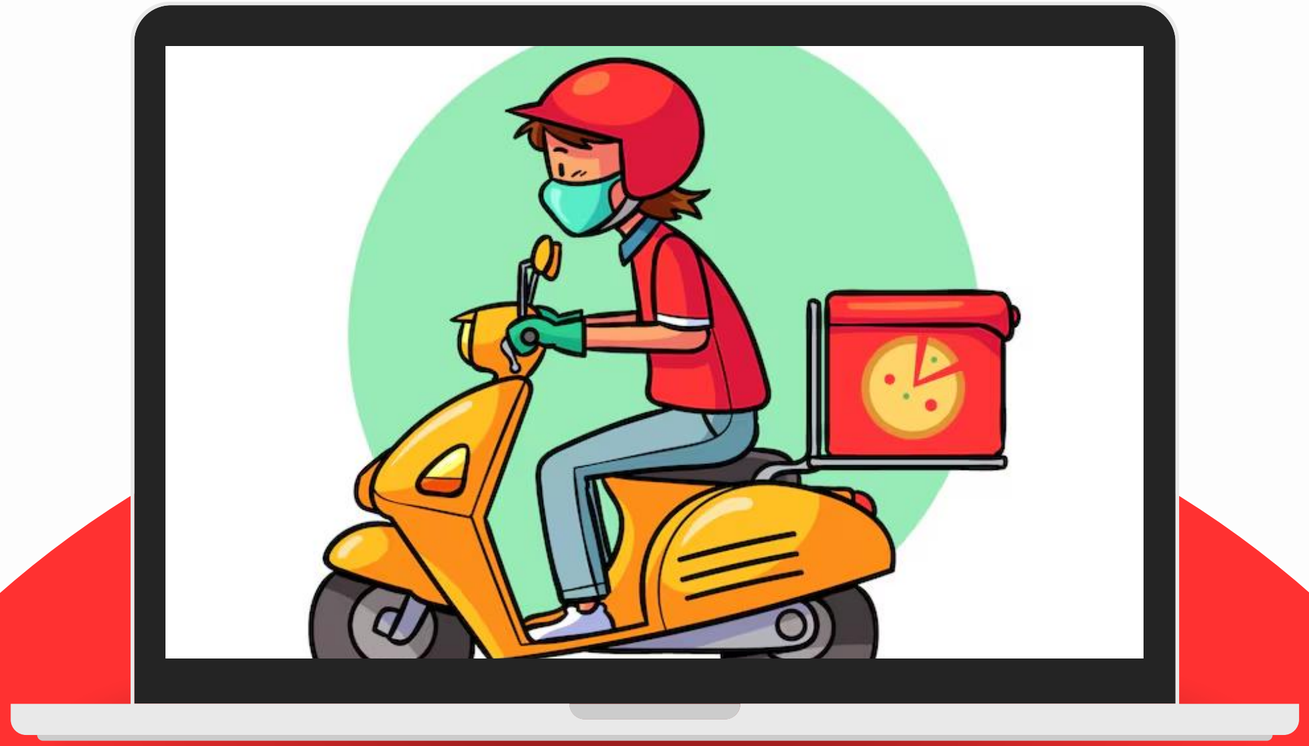
Food Delivery & Demand Analytics

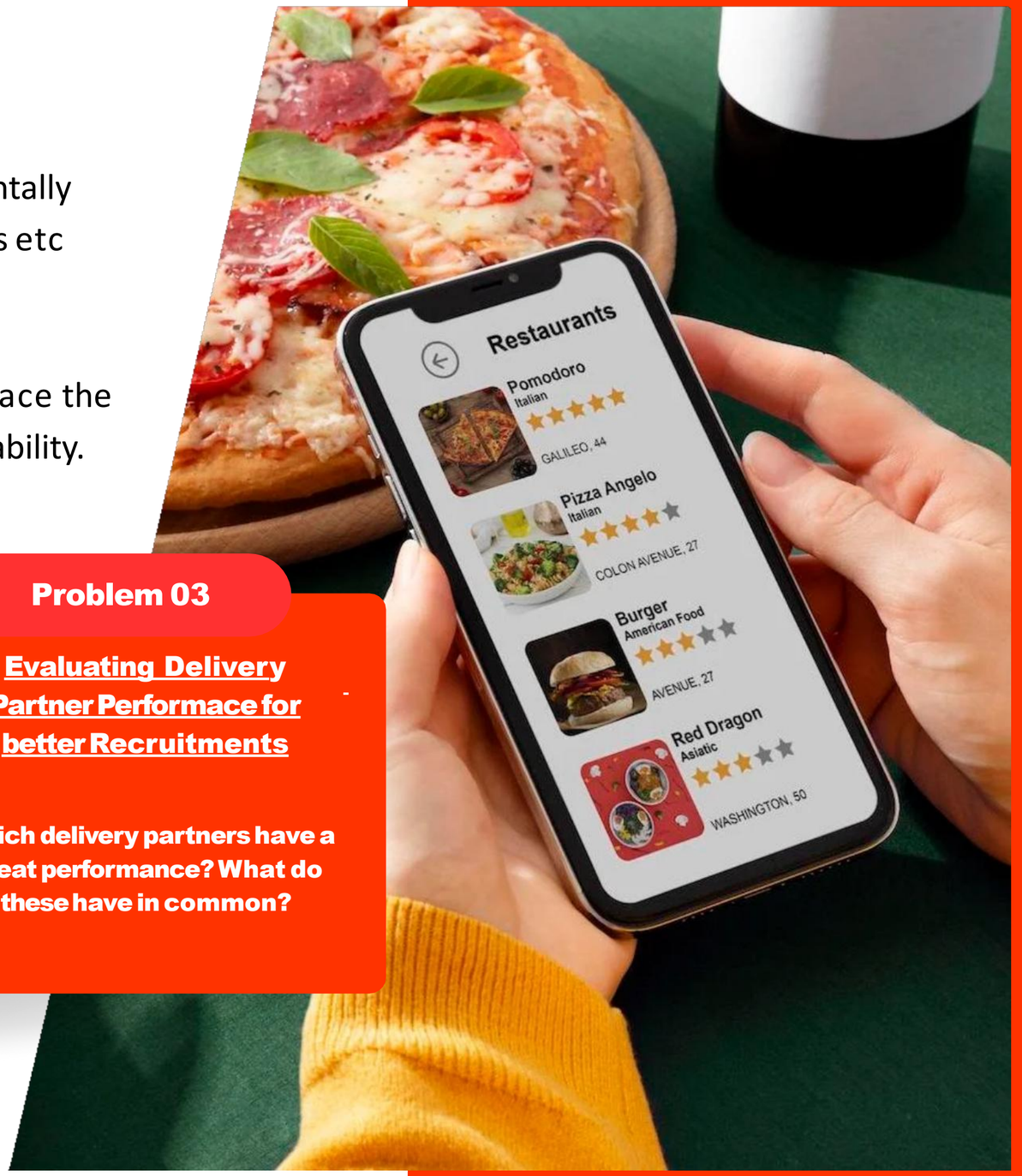


By: Gitika
20221054

A Recap of Our Problem

In the last decade, the landscape of urban dining has been fundamentally transformed, instant food delivery apps like Zomato, Swiggy, Uber Eats etc have become an almost constant part of our daily lives.

At their core, these apps are complex real time logistic networks that face the challenges of customer satisfaction operational efficiency and profitability.



Problem 01

Setting Accurate Customer Expectations

Inaccurate delivery time estimates cause customer dissatisfaction and complaints.

Problem 02

Use High Demands to ensure maximum profitability

The problem of when to apply a surge fee instaed of a fixed fee structure to ensure maximum profits.

Problem 03

Evaluating Delivery Partner Performace for better Recruitments

Which delivery partners have a great performance? What do these have in common?

The Jupyter Notebook for the Complete Final Pipeline :

https://github.com/Gitika-26/ML-Final-Project

The Dataset and Preprocessing :

Link to the Dataset

The Column in the Dataset	Description of the Column	Data Type
ID	The Order ID	Categorical
Delivery_person_ID	The Delivery Person ID	Categorical
Delivery_person_Age	The Delivery Person Age	Numerical
Delivery_person_Ratings	The Delivery Person Ratings	Numerical
Restaurant_latitude	Restaurant Latitude	Float
Restaurant_longitude	Restaurant Longitude	Float
Delivery_location_latitude	Delivery Location Latitude	Float
Delivery_location_longitude	Delivery Location Longitude	Float
Order_Date	Date of the Order	DateTime
Time_Ordered	Time of the Order	DateTime
Time_Order_picked	Pickup Time of the Order	DateTime
Weather	Weather Conditions during Order.	Categorical
Road_traffic_density	Traffic Density at the Order Time.	Categorical
Vehicle_condition	Condition of Vehicle	Numerical
Type_of_order	The Type of Food Order	Categorical
Type_of_vehicle	Type of Vehicle used for Delivery	Categorial
multiple_deliveries	Whether Drive is making multiple deliveries during the same visit ?	Binary
Festival	Whether Festive Season or Not ?	Binary
City	Type of Delivery Location	Categorical
Time taken	The Actual time taken for Delivery	Numerical

Basic Feature Engineering and Preprocessing

-> Checking for Missing Values –

There are no missing values in our dataset. We check missing values using .isnull().sum() function.

-> Encoding Categorical Variables –

The values in the categorical columns are limited in number. We use Ordinal Encoding in Scikitlearn to change those values to numerical labels like 0,1,2 …

Weather : Sunny – 0 , Windy – 1 , Stormy – 2 , Sandstorms – 3, Cloudy – 4 , Fog – 5
Traffic Density : Low – 0 , Medium – 1 , High – 2 , Jam – 3
Type of Order : Drinks – 0 , Snack – 1 , Meal – 2, Buffet – 3
Type of Vehicle : Motorcycle – 0 , Scooter – 1 , Electric Scooter – 2
City Type : SemiUrban – 0 , Urban – 1 , Metropolitian – 2

-> Transforming Columns

Spliting the Order_Date into Order_day, Order_month, Order_year, Order_day_of_week
Splitting Order Time to get Order_hour

-> Splitting and Normalising our Dataset

We use train test splitting in Scikitlearn to split our data and then we apply Standard Scaler to the features of our dataset to normalize it.

Solution One : Predicting Delivery Times using Regression

We use the following features to predict our delivery times :

Features from the Original Dataset

- Delivery_person_Age
- Delivery_person_Ratings
- Restaurant_latitude,
- Restaurant_longitude,
- Delivery_location_latitude
- Delivery_location_longitude,
- Weather
- Road_traffic_density,
- Vehicle_condition,
- Type_of_order,
- Type_of_vehicle,
- Multiple_deliveries,
- Festival, City

Features created during Preprocessing :

- Order_day
- Order_month
- Order_year
- Order_day_of_the_week
- Order_hour

Model :
Linear Regression

RMSE : 6.0934
R2 Score : 0.5754

Model :
Support Vector Regressor

RMSE : 5.6311
R2 Score : 0.6374

Model :
Decision Trees

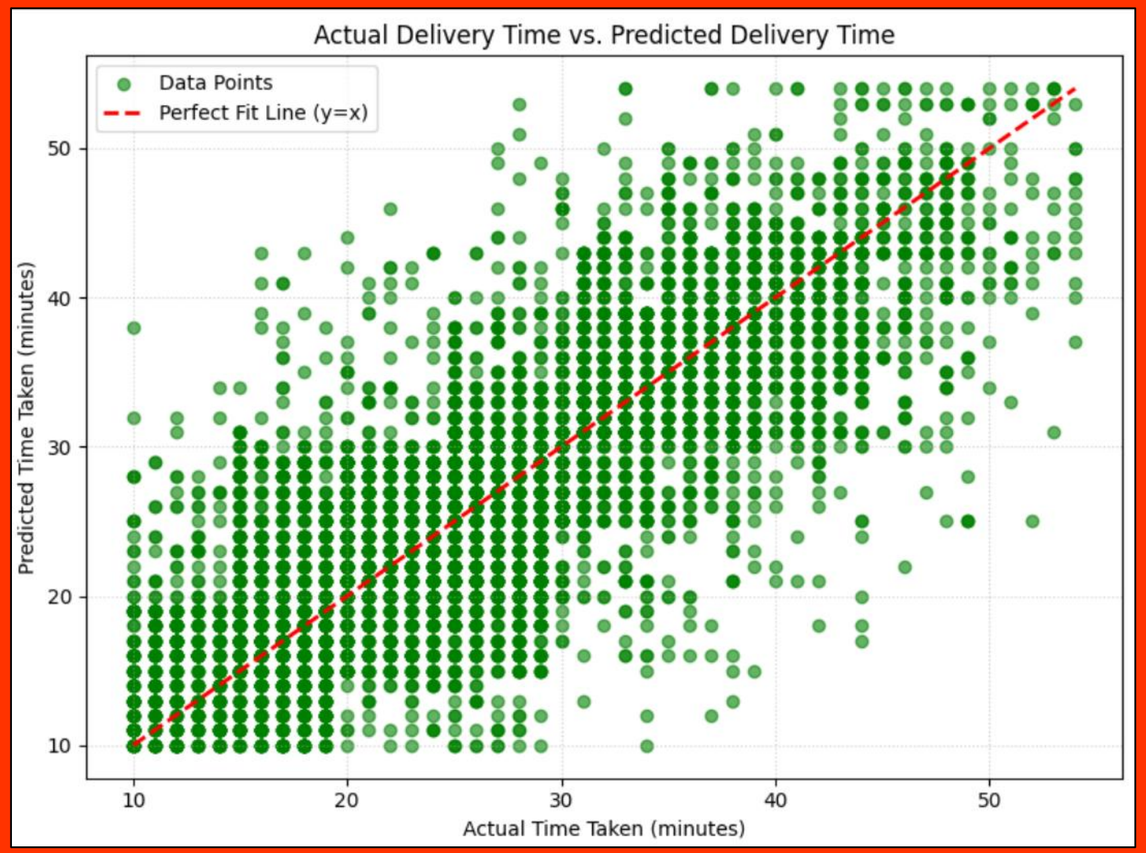
RMSE : 6.0368
R2 Score : 0.5833

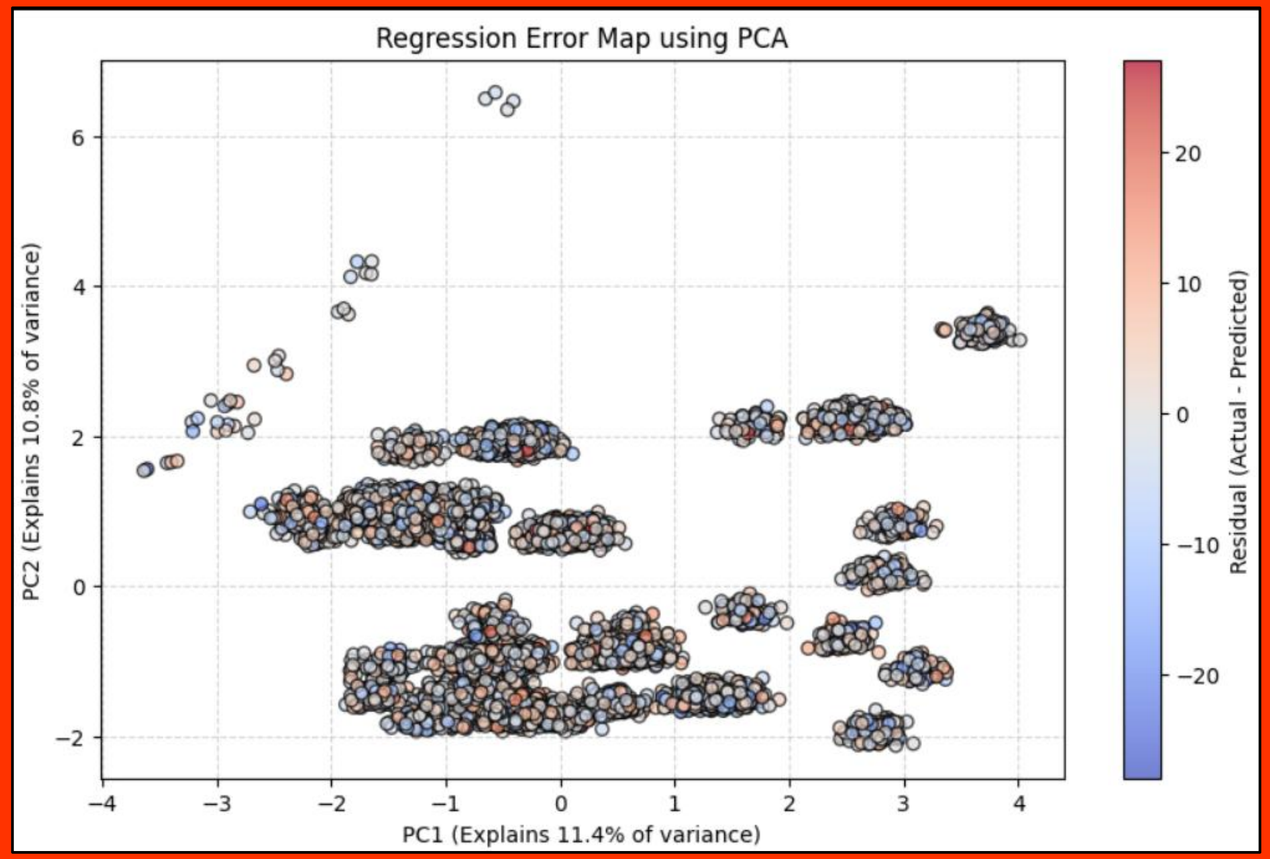
Model :
Random Forests

RMSE : 4.3954
R2 Score : 0.7791

BEST MODEL :
Random Forest

Results for Mutlivariate Linear Regression Performed :





Conclusion for Solution One : Predicting Delivery Times

- **Random Forests when trained for predicting delivery times using the features of the dataset gives 0.77 R2 score with a considerably low RMSE of 4.39.**
- **The y=x curve shows the actual vs predicted delivery times on the test set of our dataset**
- **The PCA for regression error shows that the noise in our data is random and there are no clusters with high residual errors which shows that the errors are randomly distributed, hence our model is not failing on any specific subset of the test data.**

Solution Two : When to Implement Dynamic Pricing Decisions ?

Methodology and Strategy:

The implementation follows a four-phased pipeline:

1. Spatial Segmentation - Geographic Clustering

• **Objective:** To transform delivery coordinates into operational zones.

• **Methodology:** The **K-Means Clustering** algorithm was applied to **Delivery_location_latitude** and **Delivery_location_longitude** coordinates. This partitioned the cities into distinct operational clusters (Zones), filtering out noise and invalid coordinates to ensure spatial accuracy.

2. Current Data Aggregation & Feature Engineering

•**Objective:** To convert data into a time-series format suitable for forecasting.

•**Methodology:** Data was aggregated by **Zone_ID** and **Hour**. Critical **Lag Features** were engineered, including **Demand_Last_Hour** and **Demand_Yesterday_Same_Hour**, alongside traffic data for congestion (**Traffic_Last_Hour**) to capture current dependencies and trends.

3. Demand Forecasting

•**Objective:** To estimate the volume of incoming orders for the subsequent hour.

•**Methodology:** A **Random Forest Regressor** was trained on the aggregated features. The model utilizes non-linear relationships between time, location, historical demand, and traffic conditions to output a specific **Predicted_Demand** count for each zone.

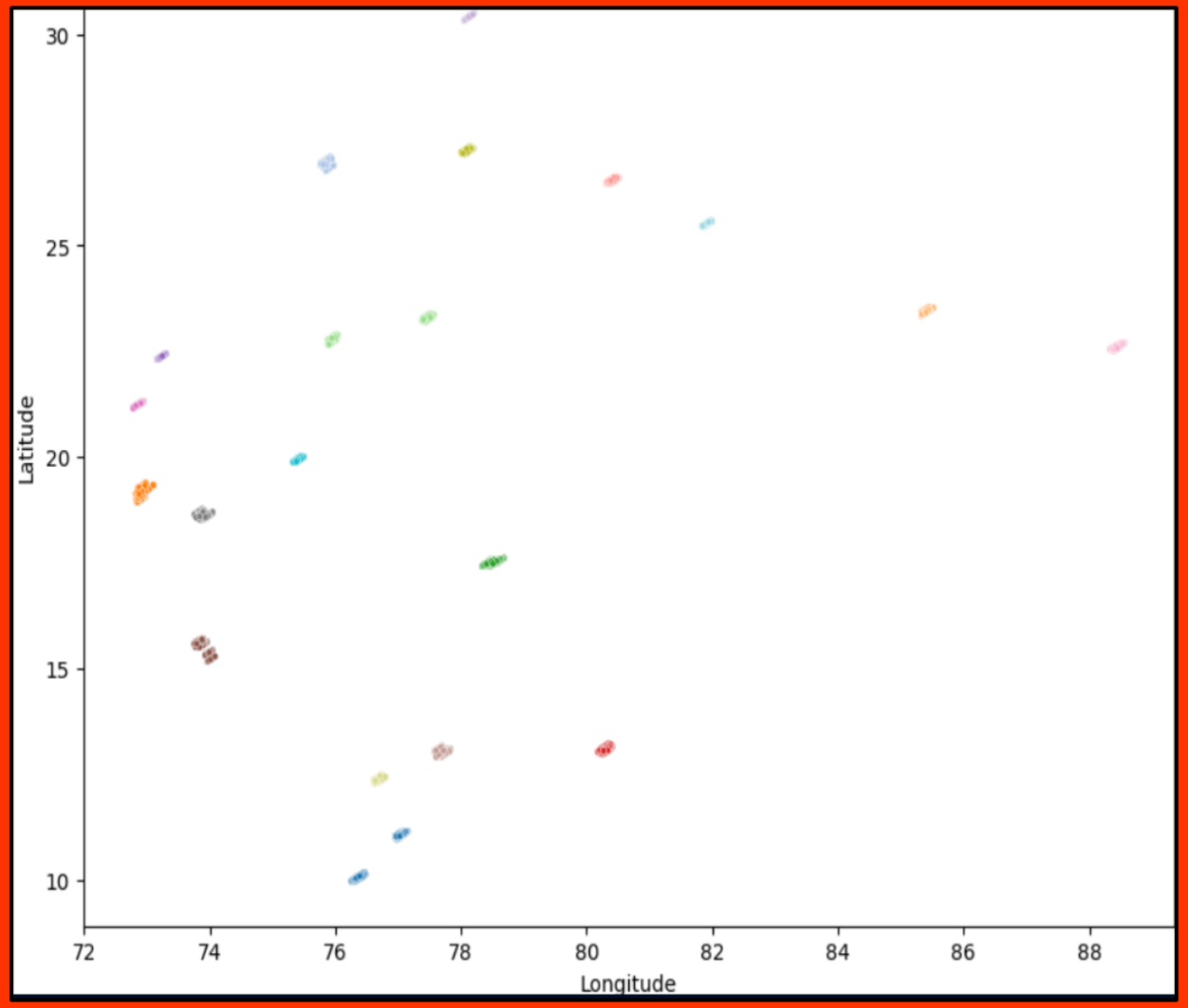
4. Dynamic Pricing Algorithm

•**Objective:** To determine the necessity and magnitude of surge fees.

•**Methodology:** A threshold-based logic was established using the **75th percentile** of historical demand per zone as the "Capacity Limit."

- **Condition:** If **Predicted_Demand** > Capacity Limit + 20%, a **1.5x multiplier** is triggered.
- **Condition:** If **Predicted_Demand** > Capacity Limit, a **1.2x multiplier** is triggered.
- **Baseline:** Otherwise, standard pricing applies.

Results for Geographical Clustering and Aggregating Lag Features



Aggregated Data Ready for Forecasting

	Zone_ID	Order_year	Order_day_of_week	Order_month	Order_day	Hour	\
392	0	2022	5	2	12	23	
457	0	2022	6	2	13	8	
458	0	2022	6	2	13	9	
459	0	2022	6	2	13	10	
460	0	2022	6	2	13	11	

	Actual_Demand	Avg_Delivery_Time	Avg_Rating	Demand_Last_Hour	\
392	11	25.181818	4.645455	9.0	
457	3	15.666667	4.666667	11.0	
458	9	19.444444	4.733333	3.0	
459	7	23.857143	4.571429	9.0	
460	6	34.166667	4.516667	7.0	

	Demand_Yesterday_Same_Hour
392	1.0
457	4.0
458	7.0
459	8.0
460	8.0

Total training samples: 5584

Results for Demand Forecasting Model and Surge Pricing Decisions :

```
Demand Forecasting Model Results
Mean Absolute Error: 2.00 orders
R2 Score: 0.4838

Prediction Snapshot
      Actual  Predicted
3910       7        6.4
1348       2        6.3
93         1        2.7
166        1        1.4
454       10        8.5
```

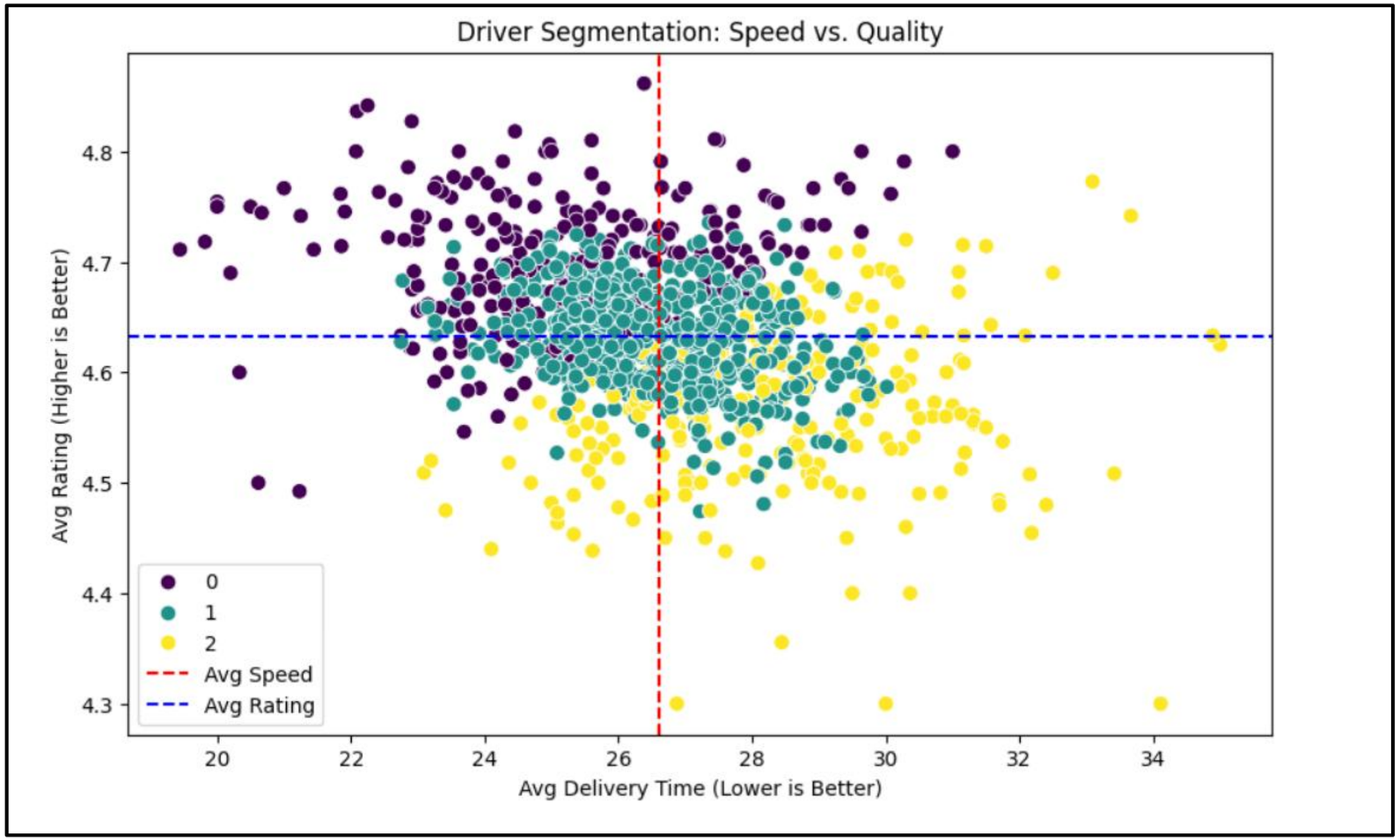
```
Surge Pricing Simulation
Total Base Revenue:     Rs.69,020.00
Total Dynamic Revenue:  Rs.76,036.00
Extra Revenue Generated: Rs.7,016.00 (+10.2%)

 Look at Changes in Price Decisions
      Zone_ID  Hour  Predicted_Demand  Surge_Multiplier
819         1    23             12.36               1.5
3981       11    22             10.80               1.5
4316       12    23              9.78               1.2
3023        6    18             10.17               1.2
494         0    22              9.17               1.2
```

Conclusion for Solution Two : Predicting Delivery Times

- **Clustered the data into geographical zones and created Zone ID for getting demand per zone.**
- **Random Forests when trained for predicting Demand Orders using the features of the aggregated dataset consisting of lag features gives an MAE of 2.0 which is considerable for forecasting**
- **We use the predicted orders to make surge fee decisions.**

Solution Three : Clustering Drivers based on Ratings and Speed to get Insights



Results of K Means Clustering applied on following features :

- **Delivery_person_Age**
- **Delivery_Person_Ratings,**
- **Time_taken**
- **No of Deliveries Made – This is created using ID counts of Delivery person in the dataset**

Conclusions

- We implement our proposed solutions using a full ML pipeline of Data Preprocessing - Handling Categorical Variables, Creating New Features using Feature Engineering, Aggregating relevant features into a new Dataframe.
- For the Regression tasks – Random Forests Model gives the best R2 score
- For Dynamic Price Decisions : We implement clustering for geographical locations and then create lag features and using them to predict the order for the next hour and then using them to make Dynamic Pricing Decisions.
- For Driver Performance Insights, we use clustering to find insights about good driver performances